DBMS -- SQL Traps (8 Must-Know)

Format: Rule + Wrong query + Correct query + Follow-up.
Target: Explain any trap in 40s with 1 example cold.

TRAP 1 -- COUNT(*) vs COUNT(col)
Rule: COUNT(*) counts every row including NULLs. COUNT(col) counts only non-NULL values.

Function	NULLs	Returns
COUNT(*)	Included	Total row count
COUNT(col)	Skipped	Non-NULL value count
COUNT(DISTINCT col)	Skipped	Unique non-NULL count

Table: users

id	manager_id
1	5
2	NULL
3	5

sql
```
SELECT COUNT(*)          FROM users;  -- 3  (all rows)
SELECT COUNT(manager_id) FROM users;  -- 2  (NULL skipped)
SELECT COUNT(DISTINCT manager_id) FROM users; -- 1  (only unique: 5)
```
Fix line: Use COUNT(*) for total rows. Use COUNT(col) when NULLs should not count.
Follow-up: "What does COUNT(DISTINCT col) do?"

Counts unique non-NULL values only.

TRAP 2 -- LEFT JOIN becomes INNER JOIN because of WHERE on right table
Rule: A WHERE filter on the right-table column drops unmatched rows -- turning LEFT JOIN into INNER JOIN silently.
Wrong:
sql
```
SELECT u.id, d.name
FROM users u
LEFT JOIN dept d ON d.id = u.dept_id
WHERE d.name = 'HR';
```

-- Users with no dept have d.name = NULL
-- NULL = 'HR' is FALSE those rows dropped
-- Result: behaves like INNER JOIN
Correct:
sql
SELECT u.id, d.name
FROM users u
LEFT JOIN dept d
  ON d.id = u.dept_id
  AND d.name = 'HR';
-- Filter is part of JOIN condition
-- Unmatched users still appear with NULL dept
One-liner: "Filter the right table in ON -- not WHERE -- if you want to keep unmatched left rows."
Follow-up: "When is WHERE on right-table column intentional?"

When you actually want INNER JOIN behaviour -- i.e., only want matched rows.

TRAP 3 -- NOT IN + NULL (3-valued logic)
Rule: If the subquery returns even one NULL, NOT IN returns zero rows -- always.
Wrong:
sql
-- manager_id has NULLs entire result disappears
SELECT * FROM users
WHERE id NOT IN (SELECT manager_id FROM users);
Why it breaks:
SQL uses 3-valued logic: TRUE / FALSE / UNKNOWN
1 NOT IN (5, NULL):
  1 != 5   TRUE
  1 != NULL   UNKNOWN   one UNKNOWN poisons the whole check row dropped
Correct:
sql
SELECT * FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM users m WHERE m.manager_id = u.id
);
One-liner: "Never use NOT IN when the subquery column can have NULLs. Use NOT EXISTS."
Follow-up: "What is 3-valued logic in SQL?"

SQL has TRUE / FALSE / UNKNOWN. Any comparison with NULL gives UNKNOWN. Only TRUE passes a WHERE filter.

TRAP 4 -- WHERE vs HAVING
Rule:
- WHERE filters rows BEFORE GROUP BY -- no aggregate functions allowed here
- HAVING filters groups AFTER aggregation -- aggregate functions allowed
SQL Execution Order:
FROM JOIN WHERE GROUP BY HAVING SELECT ORDER BY
Wrong:
sql
SELECT dept_id, COUNT(*)
FROM users
WHERE COUNT(*) >= 5   -- ERROR: aggregate not allowed in WHERE
GROUP BY dept_id;
Correct:
sql
SELECT dept_id, COUNT(*) AS headcount
FROM users
WHERE active = true   -- row-level filter BEFORE grouping
GROUP BY dept_id
HAVING COUNT(*) >= 5;   -- group-level filter AFTER aggregation
Your 20-second script:

"WHERE runs before GROUP BY -- filters rows, no aggregates allowed. HAVING runs after GROUP BY -- filters groups, aggregates allowed. Example: WHERE removes inactive rows first, then HAVING COUNT >= 5 keeps only large departments."

Follow-up: "Can you use column aliases in HAVING?"

Depends on DB. MySQL allows it; PostgreSQL does not -- you must repeat the full expression.

TRAP 5 -- NULL comparisons are not normal
Rule: NULL = NULL is NOT TRUE -- it returns UNKNOWN. Always use IS NULL / IS NOT NULL.

Wrong:
sql
SELECT * FROM users WHERE manager_id = NULL;   -- 0 rows always
SELECT * FROM users WHERE manager_id != NULL; -- 0 rows always
Correct:
sql
SELECT * FROM users WHERE manager_id IS NULL;
SELECT * FROM users WHERE manager_id IS NOT NULL;
NULL arithmetic traps:
sql
SELECT NULL + 5;   -- NULL (any math with NULL = NULL)
SELECT NULL OR TRUE;   -- TRUE (OR exception)
SELECT NULL AND FALSE; -- FALSE (AND exception)
Follow-up: "What is COALESCE?"

COALESCE(col, default) returns the first non-NULL value. Use to safely replace NULLs in output.

TRAP 6 -- Double-counting due to JOIN multiplication
Rule: A 1-to-many JOIN duplicates the "one" side's rows. SUM/COUNT on that side gets inflated.
Scenario: 1 Order has 3 line items after JOIN, order amount counted 3 times.
Wrong:
sql
SELECT o.customer_id, SUM(o.amount)
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
GROUP BY o.customer_id;
-- Order 101 with amount=1000 and 3 items SUM = 3000 (wrong!)
Correct -- pre-aggregate:
```sql
-- Option 1: deduplicate before aggregating
SELECT o.customer_id, SUM(o.amount)
FROM orders o
JOIN (SELECT DISTINCT order_id FROM order_items) oi
  ON oi.order_id = o.id
GROUP BY o.customer_id;
-- Option 2: filter without joining
SELECT customer_id, SUM(amount)
FROM orders
WHERE id IN (SELECT DISTINCT order_id FROM order_items)
GROUP BY customer_id;
```
One-liner: "When joining one-to-many, ask: will this inflate my aggregates? Pre-aggregate the many side first."
Follow-up: "Quick fix using DISTINCT?"

COUNT(DISTINCT o.id) prevents counting the same order multiple times.

TRAP 7 -- UNION vs UNION ALL
Rule:
- UNION deduplicates results (hidden sort internally slower)
- UNION ALL keeps all rows including duplicates (no sort faster)
```sql
-- UNION: deduplicates, costs a sort
SELECT id FROM active_users
UNION
SELECT id FROM premium_users;
-- UNION ALL: no dedup, faster
SELECT id FROM active_users
UNION ALL
SELECT id FROM premium_users;
```
When to use which:

Scenario
Use

Sources have no overlapping rows
UNION ALL

Need unique combined results
UNION

Building logs / history (duplicates valid)
UNION ALL

Showing combined unique user list
UNION

Interview line: "Default to UNION ALL. Only use UNION when you explicitly need deduplication -- UNION does a hidden sort that costs performance."
Follow-up: "What must both queries in a UNION share?"

Same number of columns and compatible data types in each position.

TRAP 8 -- AND/OR Precedence (missing parentheses)
Rule: AND binds tighter than OR. SQL evaluates A OR B AND C as A OR (B AND C) -- almost never what you intended.
Precedence order: NOT > AND > OR
Wrong (ambiguous):
sql
SELECT * FROM users
WHERE role = 'admin' OR status = 'active' AND city = 'Delhi';
-- SQL reads: role='admin' OR (status='active' AND city='Delhi')
-- All admins from any city slip through!
Correct (explicit):
```sql
-- If intent is: (admin or active) AND must be in Delhi
SELECT * FROM users
WHERE (role = 'admin' OR status = 'active') AND city = 'Delhi';
-- If intent is: admin from anywhere OR active person from Delhi
SELECT * FROM users
WHERE role = 'admin' OR (status = 'active' AND city = 'Delhi');
```
One-liner: "Always parenthesize mixed AND/OR conditions. Never trust implicit precedence."
Follow-up: "Does NOT have higher precedence than AND?"

Yes. Full order: NOT > AND > OR.

Quick-Reference Cheat Sheet
TRAP 1 COUNT(*) = rows; COUNT(col) = non-NULLs only
TRAP 2 WHERE on right table kills LEFT JOIN move filter to ON
TRAP 3 NOT IN + NULL subquery always empty; use NOT EXISTS
TRAP 4 WHERE = before GROUP BY (no aggregates); HAVING = after (aggregates ok)
TRAP 5 NULL = NULL is UNKNOWN; always use IS NULL / IS NOT NULL
TRAP 6 1-to-many JOIN inflates SUM/COUNT; pre-aggregate or use DISTINCT
TRAP 7 UNION deduplicates (slow); UNION ALL keeps dupes (fast); default ALL
TRAP 8 AND > OR precedence; always parenthesize mixed conditions

Mini-Mock -- 8 Rapid-Fire Questions (40s each)

"Difference between COUNT() and COUNT(col)? Give an example where they return different values."*
"Why can LEFT JOIN + WHERE right_table.col = value be wrong? How do you fix it?"
"Why is NOT IN dangerous when the subquery column has NULLs? What is the safe alternative?"
"WHERE vs HAVING -- one-sentence difference + one example query."
"Why does WHERE col = NULL return zero rows? What do you write instead?"
"Give an example of JOIN double-counting inflating a SUM. How do you fix it?"
"UNION vs UNION ALL -- when to use each? Which is faster and why?"
"What does A OR B AND C evaluate to in SQL? How do you make intent explicit?"

Say each answer out loud. Target: rule in 1 line + example in 30s + follow-up handled in 10s.